

Министерство образования и науки Российской Федерации
Московский физико-технический институт
(национальный исследовательский университет)

Физтех-школа прикладной математики и информатики
Кафедра системного программирования (ИСП РАН)

Выпускная квалификационная работа бакалавра

Исследование и разработка расширяемой системы генерации событий для систем глубокого анализа сетевого трафика

Автор:

Студент Б05-232 группы
Платонов Тимофей Александрович

Научный руководитель:

канд. физ.-мат. наук
Гетьман Александр Игоревич



Москва 2026

Содержание

1	Введение	4
2	Обзор существующих решений	6
2.1	Анализаторы сетевого трафика	6
2.1.1	ntopng	6
2.1.2	Suricata	6
2.1.3	Zeek	7
2.1.4	Итоговый анализ анализаторов	7
2.2	Библиотеки систем событий	7
2.2.1	Boost.Signals2	7
2.2.2	Eventpp	8
2.2.3	Sigslot	8
2.2.4	Итоговый анализ библиотек событий	8
3	Построение решения и исследование задачи	10
3.1	Выбор подходящих инструментов	10
3.2	Концептуальная модель системы событий	10
3.3	Механизм расширения через наследование	11
3.4	Проблема хранения разнотипных обработчиков	11
3.5	Идентификация типов событий без RTTI	13
3.5.1	Решение на основе CRTP и атомарного генератора идентификаторов	13
3.5.2	Сравнение производительности стратегий идентификации типов .	15
3.6	Механизм диспетчеризации событий	15
3.6.1	Разделение стратегий выполнения	15
3.6.2	Интерфейс регистрации обработчиков	17
3.7	Сравнительный анализ стратегий выполнения обработчиков	17
3.7.1	Методология эксперимента	17
3.7.2	Результаты измерения задержек	18
3.7.3	Влияние на пропускную способность системы	19
3.7.4	Практические рекомендации для сценариев DPI	19
3.7.5	Выводы по разделу	20
4	Экспериментальные исследования и оценка эффективности	22
4.1	Описание тестового стенда	22
4.1.1	Аппаратная конфигурация	22
4.1.2	Программное окружение	22
4.1.3	Меры обеспечения воспроизводимости	23
4.2	Методология измерения производительности	23
4.3	Базовая производительность диспетчеризации	24
4.4	Производительность с обработчиками	24

4.4.1	Синхронные обработчики: масштабирование по потокам	24
4.4.2	Влияние числа обработчиков на задержку	25
4.4.3	Сравнение синхронного и асинхронного выполнения	25
4.5	Сравнение с альтернативными решениями	26
4.6	Профилирование использования ресурсов	26
4.7	Ограничения исследования	26
4.8	Выводы по главе	27
5	Заключение	29

1 Введение

Введение

С каждым годом интернет-инфраструктура занимает всё более значимую позицию в функционировании государственных, коммерческих и персональных информационных систем. Экспоненциальный рост объёмов передаваемого трафика, обусловленный распространением облачных сервисов, мультимедийных потоков, IoT-устройств и распределённых вычислительных кластеров, формирует беспрецедентную нагрузку на сетевые подсистемы. В этих условиях традиционные методы поверхностного анализа заголовков пакетов (shallow packet inspection) оказываются недостаточными для обеспечения требуемого уровня наблюдаемости, безопасности и соответствия регуляторным нормам. Необходимость перехода к технологиям глубокого анализа сетевого трафика (Deep Packet Inspection, DPI) [1] определяется требованием извлечения семантических признаков на уровне приложений, реконструкции сессий и контекстной интерпретации потоков данных.

Технология DPI подразумевает полный анализ структуры пакета, включая заголовки канального, сетевого, транспортного и прикладного уровней модели OSI. В отличие от методов, оперирующих исключительно метаданными потока, DPI-анализаторы осуществляют парсинг полезных данных, идентификацию протоколов (в том числе зашифрованных и обфусцированных), извлечение полей заголовков прикладного уровня и формирование структурированного представления передаваемой информации. Типичный конвейер DPI состоит из этапов захвата кадров, дефрагментации, реконструкции TCP-потоков, диссекции протоколов и сохранения извлечённых полей в нормализованном формате. Однако полученный формат хранения, оптимизированный для эффективного разбора и компактного размещения в памяти, часто оказывается непрактичным для последующего анализа, корреляции и интеграции с системами мониторинга. Сырые структуры данных избыточны и требуют дополнительной трансформации перед использованием аналитическими модулями или системами класса SIEM [2].

Для решения указанной проблемы применяется событийно-ориентированная архитектура (Event-Driven Architecture, EDA), в которой промежуточным звеном между низкоуровневым парсером трафика и высокоуровневыми анализаторами выступает система событий. Система событий представляет собой набор механизмов, оперирующих двумя базовыми сущностями: событиями и обработчиками. Событие определяется как типизированный набор данных, фиксирующий факт возникновения определённого состояния или действия в сетевом стеке (например, “установлено TLS-соединение”, “обнаружен аномальный DNS-запрос”, “превышен порог пропускной способности для потока”). Обработчик выступает в роли подписчика, принимающего события заданного типа и выполняющего детерминированную логику: агрегацию метрик, фильтрацию, обогащение контекстом или передачу во внешние хранилища. Ключевым требованием к такой системе является возможность гибкой настройки схемы событий, динамической

регистрации обработчиков, обеспечения типобезопасности и предсказуемой производительности в условиях высокой нагрузки.

Существующие промышленные и академические решения в области анализа трафика часто реализуют жёстко закодированные или скриптовно-зависимые механизмы генерации событий, что ограничивает их расширяемость, увеличивает задержки при обработке и усложняет интеграцию в существующие инфраструктуры. Отсутствие унифицированного, высокопроизводительного и типобезопасного программного компонента, специально спроектированного для встраивания в DPI-конвейеры, обуславливает актуальность настоящего исследования. Целью работы является изучение, проектирование и разработка библиотеки, предоставляющей функционал расширяемой системы генерации событий, пригодной для интеграции в различные анализаторы трафика. Для достижения цели решаются следующие задачи: анализ архитектурных паттернов современных DPI-систем и существующих библиотек событий; формулировка функциональных и нефункциональных требований; проектирование API и внутренней архитектуры с учётом требований к масштабируемости и низкой задержке; реализация прототипа на языке C++; экспериментальная оценка производительности, устойчивости и расширяемости разработанного решения.

2 Обзор существующих решений

2.1 Анализаторы сетевого трафика

Современные системы анализа сетевого трафика можно классифицировать по глубине инспектирования, архитектуре обработки и модели расширения функциональности. В данном подразделе рассматриваются три наиболее распространённых открытых решения: ntopng, Suricata и Zeek, с акцентом на их подходы к генерации и обработке событий.

2.1.1 ntopng

Библиотека ntopng представляет собой систему мониторинга сетевого трафика, ориентированную на визуализацию потоков, анализ пропускной способности и выявление аномалий на основе статистических моделей. Архитектура ntopng базируется на потоковом анализе (flow-based), где пакеты агрегируются в записи NetFlow/IPFIX с последующим хранением в Redis или базе данных TimescaleDB. Генерация событий в ntopng реализована преимущественно через пороговые механизмы и интеграцию с внешними системами оповещения. Расширяемость обеспечивается через плагины на языке Lua, однако внутренняя событийная шина не выделена в отдельный компонент, что ограничивает гибкость маршрутизации событий и требует перезапуска процессов при глубокой модификации логики обработки. Система эффективна для задач мониторинга производительности, но не предназначена для детального анализа протоколов прикладного уровня в реальном времени [3].

2.1.2 Suricata

Suricata является многопоточным IDS/IPS-движком, поддерживающим сигнатурный анализ, обнаружение аномалий и глубокую инспекцию пакетов. Архитектура Suricata построена на основе конвейера обработки, где каждый пакет проходит через стадии дефрагментации, реконструкции потоков и применения правил. События генерируются при совпадении сигнатур или нарушении правил безопасности и выводятся в форматах eve.json или syslog. Расширение функциональности возможно через добавление новых правил в формате YAML/Lua и подключение внешних модулей, однако базовый механизм событийного движка жёстко привязан к ядру обработки. Динамическое добавление новых типов событий требует компиляции или перезапуска, а обработка событий осуществляется синхронно в рамках потока обработки пакетов, что создаёт узкие места при высокой нагрузке. Suricata демонстрирует высокую пропускную способность, но её событийная модель не оптимизирована для гибкой настройки пользовательского анализа без вмешательства в исходный код [4].

2.1.3 Zeek

Zeek (ранее Bro) позиционируется как система мониторинга сетевой безопасности, ориентированная на протокольный анализ и генерацию структурированных логов. В отличие от сигнатурных систем, Zeek использует событийно-ориентированную модель, где каждый этап обработки пакета (установление соединения, запрос/ответ протокола, завершение сессии) генерирует соответствующее событие. Обработка событий осуществляется через встроенный скриптовый язык ZeekScript, позволяющий описывать логику анализа без модификации ядра. Однако интерпретация скриптов вносит значительные накладные расходы, а управление состоянием событий ограничено однопоточной или слабо масштабируемой моделью. Для внесения изменений в схему событий или добавления новых обработчиков часто требуется перезагрузка политик или перезапуск процесса, что снижает доступность в режимах непрерывного мониторинга. Zeek остаётся эталоном в области разбора сетевых протоколов, но её событийная подсистема не удовлетворяет требованиям к высокой пропускной способности и динамической расширяемости в современных распределённых средах [5].

2.1.4 Итоговый анализ анализаторов

Сравнительный анализ показывает, что существующие системы либо фокусируются на потоковой статистике (ntopng), либо на сигнатурном обнаружении (Suricata), либо на скриптовой событийной обработке (Zeek). Ни одно из решений не предоставляет выделенной, высокопроизводительной и типобезопасной библиотеки событий, предназначенной исключительно для встраивания в сторонние DPI-конвейеры. Ограничения включают отсутствие динамического реестра схем событий, синхронную обработку с блокирующими вызовами, необходимость перезапуска для обновления логики и слабую изоляцию пользовательских обработчиков. Эти недостатки формируют технологический пробел, который заполняется разработкой независимой расширяемой системы генерации событий.

2.2 Библиотеки систем событий

2.2.1 Boost.Signals2

Boost.Signals2 представляет собой библиотеку для реализации паттерна “наблюдатель” с поддержкой многопоточности и управления временем жизни подключений [6]. Библиотека обеспечивает типобезопасное соединение сигналов и слотов через механизм function objects.

Ключевыми преимуществами Boost.Signals2 являются зрелость реализации, кроссплатформенность и интеграция с экосистемой Boost. Однако библиотека демонстрирует значительные накладные расходы на диспетчеризацию (50-100 нс на вызов обработчика) из-за использования `std::function` и динамического выделения памяти для хранения подключений [7].

Для сценариев DPI с требованиями к задержке обработки на уровне единиц наносекунд накладные расходы Boost.Signals2 являются неприемлемыми, особенно при большом числе обработчиков на событие.

2.2.2 Eventpp

Eventpp — современная библиотека обработки событий для C++, ориентированная на производительность и типобезопасность [8]. Библиотека использует шаблоны для генерации кода диспетчеризации на этапе компиляции, что позволяет избежать накладных расходов type erasure.

Eventpp поддерживает различные стратегии выполнения обработчиков, включая синхронный вызов и асинхронную очередь. Однако библиотека не предоставляет встроенной интеграции с асинхронными runtime, что ограничивает её применимость для гибридных сценариев обработки.

Кроме того, шаблонная природа Eventpp приводит к увеличению времени компиляции и размера бинарного файла при большом числе типов событий, что может быть критичным для встраиваемых систем.

2.2.3 Sigslot

Sigslot — легковесная библиотека паттерна “сигнал-слот” с типобезопасной маршрутизацией и минимальными зависимостями [9]. Библиотека обеспечивает только синхронный вызов обработчиков.

Однако для задач DPI библиотека не подходит по следующим причинам: отсутствие встроенной асинхронной диспетчеризации требует ручной интеграции с внешними планировщиками; синхронная модель обработки блокирует поток-источник событий; отсутствие приоритизации обработчиков снижает гибкость при пиковых нагрузках; динамическое выделение памяти для слушателей приводит к фрагментации и промахам кэша.

2.2.4 Итоговый анализ библиотек событий

Анализ существующих библиотек обработки событий позволяет сформулировать требования к целевому решению:

Таблица 1 – Сравнительный анализ библиотек событий

Критерий	Boost.Signals2	Eventpp	Sigslot
Накладные расходы на вызов	50-100 нс	30-50 нс	10-30 нс
Поддержка async runtime	Ограниченная	Отсутствует	Отсутствует
Типобезопасность	Высокая	Очень высокая	Высокая
Гибкость выполнения	Низкая	Средняя	Средняя
Интеграция с DPI	Низкая	Средняя	Средняя

Ни одна из рассмотренных библиотек не удовлетворяет одновременно всем требованиям современных DPI-систем: минимальные накладные расходы, гибридная модель выполнения, типобезопасность и гибкость расширения. Это обосновывает необходимость разработки специализированного решения.

3 Построение решения и исследование задачи

3.1 Выбор подходящих инструментов

Из-за огромных и постоянно растущих объёмов трафика разрабатываемая система должна обладать высокой пропускной способностью, низкой и предсказуемой задержкой, эффективным использованием многопроцессорных архитектур и минимальными накладными расходами на управление памятью. Языки программирования с динамической типизацией и автоматическим контролем памяти (сборкой мусора) не подходят для данных требований, поскольку недетерминированные паузы GC, overhead runtime и отсутствие контроля над расположением данных в памяти приводят к высоким накладным расходам и деградации производительности при пиковых нагрузках [10].

В качестве основного языка разработки выбран C++ (стандарт C++17/20), обеспечивающий статическую типобезопасность, zero-cost абстракции, детерминированное управление ресурсами и прямой доступ к аппаратным возможностям процессора. Инструментарий включает систему сборки CMake для управления зависимостями и кроссплатформенной компиляции, Git для контроля версий, Clang/GCC в качестве компиляторов с поддержкой оптимизаций уровня -O3 и LTO, а также AddressSanitizer и ThreadSanitizer для выявления ошибок управления памятью и состояний гонки на ранних этапах разработки. Выбор C/C++ обоснован его доминированием в высокопроизводительных сетевых системах (DPDK, Suricata, Zeek core) и наличием зрелой экосистемы шаблонного метапрограммирования, позволяющей реализовать событийную модель без лишних накладных расходов [11].

3.2 Концептуальная модель системы событий

Разрабатываемая система событий строится на двух фундаментальных абстракциях: *событии* (event) и *обработчике* (handler). Событие представляет собой иммутабельный контейнер данных, описывающий произошедший факт в предметной области — например, приём сетевого пакета, завершение анализа протокола или обнаружение аномалии. Обработчик — это функция, выполняющая произвольную логику в ответ на событие определённого типа [12].

Формально, система событий определяет интерфейс для:

- регистрации обработчиков для типов событий;
- публикации событий с автоматической диспетчеризацией к соответствующим обработчикам;
- управления временем жизни данных и стратегиями выполнения (синхронно/асинхронно).

Ключевым требованием к системе является **гибкость расширения**: пользователь должен иметь возможность определять произвольные типы событий и соответствующие им обработчики без модификации ядра библиотеки. Это требование противоречит традиционным подходам, основанным на статической типизации и компиляции с закрытым набором типов.

3.3 Механизм расширения через наследование

Для обеспечения расширяемости при сохранении типобезопасности на этапе компиляции предложен механизм наследования от шаблонного базового класса `EventBase`. Пользователь определяет новый тип события следующим образом:

Листинг 1: Определение пользовательского события через наследование от `EventBase`

```
1 struct PacketEvent : public Core::event::EventBase<PacketEvent> {  
2     uint32_t src_ip;  
3     uint32_t dst_ip;  
4     uint16_t src_port;  
5     uint16_t dst_port;  
6     uint8_t  protocol;  
7 };
```

Использование **CRTP** (Curiously Recurring Template Pattern) [13] позволяет базовому классу обращаться к производному типу на этапе компиляции, избегая накладных расходов виртуальных функций. Шаблонный параметр `Derived` в `EventBase<Derived>` обеспечивает статическую связь между базовым интерфейсом и конкретной реализацией события. В дальнейшем это потребуется для реализации идентификатора типа для события.

3.4 Проблема хранения разнотипных обработчиков

Система должна поддерживать регистрацию произвольных обработчиков для каждого типа события. Обработчик может быть:

- обычной функцией с сигнатурой `void(const EventType&);`
- лямбда-выражением с захватом переменных;
- функциональным объектом (функтором) с состоянием;
- асинхронной корутиной (в перспективе).

Для хранения таких разнотипных объектов в едином контейнере необходим механизм **type erasure** — стирания информации о конкретном типе с сохранением интерфейса вызова [14]. Стандартная библиотека C++ предоставляет `std::function` для этой цели, однако его реализация всегда использует динамическое выделение памяти, что неприемлемо для высокопроизводительных сценариев.

В данной работе используется библиотека `Naios/function2` [15], предоставляющая `unique_function` с поддержкой **small buffer optimization** (SBO) [16].

Листинг 2: Handler с type erasure

```
1 namespace Core::event {
2
3 class Handler {
4 public:
5     template <typename EventType, typename Callable>
6     Handler(std::type_identity<EventType>, Callable&& callback) {
7         ...
8     }
9
10    void handle(const void* event) {
11        ...
12    }
13
14 private:
15     using InvokeFunction = fu2::unique_function<void(const void*)>;
16     InvokeFunction invoke_{};
17 };
18
19 } // namespace Core::event
```

Использование `std::type_identity` в конструкторе позволяет явно указать тип события при регистрации обработчика, избегая неоднозначностей вывода шаблонов компилятором.

Листинг 3: Шаблонный конструктор для Handler

```
1 Handler(std::type_identity<EventType>, Callable&& callback) {
2     static_assert(
3         std::is_invocable_v<Callable, const EventType>,
4         "Callback must accept const EventType&"
5     );
6
7     invoke_ = [cb = std::forward<Callable>(callback)](const void* event) {
8         cb(*static_cast<const EventType*>(event));
9     };
10 }
```

В конструкторе происходит статическая проверка того, что переданный обработчик правильного типа, и сохранение его в `invoke_` через лямбда-обёртку, внутри которой выполняется безопасное приведение `void*` к конкретному типу события и вызов оригинального обработчика.

3.5 Идентификация типов событий без RTTI

В основе механизма диспетчеризации событий лежит задача маршрутизации: в момент публикации события система должна определить, какие именно обработчики были зарегистрированы для данного типа, и корректно их вызвать. Поскольку обработчики различных типов событий хранятся в структурированном контейнере, необходим эффективный механизм сопоставления типа публикуемого события с соответствующим набором функций. Для решения этой задачи каждому типу события ставится в соответствие уникальный числовой идентификатор, выступающий в роли ключа поиска. Наличие такого идентификатора позволяет реализовать операцию выборки обработчиков за константное время $O(1)$ через прямую индексацию в массиве, исключая необходимость последовательного перебора всех зарегистрированных подписок или выполнения ресурсоёмких операций сравнения типов непосредственно в критическом пути обработки.

Стандартные механизмы C++ предоставляют два подхода:

1. **RTTI (Run-Time Type Information)** через `typeid(T).hash_code()`. Недостатки: требует включения флага `-frtti`, имеет высокие накладные расходы [17] и зависит от реализации компилятора.
2. **Виртуальные функции** с возвращением идентификатора. Недостатки: требование виртуального деструктора, накладные расходы на косвенный вызов через `vtable` (5–10 нс) и увеличение размера объекта на указатель.

Оба подхода вносят неприемлемые накладные расходы для сценариев DPI, где диспетчеризация события должна выполняться за единицы наносекунд.

3.5.1 Решение на основе CRTP и атомарного генератора идентификаторов

Предложен механизм генерации уникальных идентификаторов типов на этапе выполнения с нулевыми накладными расходами в горячем пути:

Листинг 4: Реализация EventBase с CRTP и атомарным генератором идентификаторов

```
1 namespace Core::event {
2
3 class IdGenerator {
4 public:
5     static size_t nextId() {
6         static std::atomic<size_t> id_generator_{0};
7         return id_generator_.fetch_add(1, std::memory_order::relaxed);
8     }
9 };
10
11 template <typename Derived>
12 struct EventBase : private IdGenerator {
```

```

13 static size_t getId() noexcept {
14     static const size_t type_id = nextId();
15     return type_id;
16 }
17 };
18
19 } // namespace Core::event

```

Ключевые особенности реализации:

- **Атомарный счётчик** `std::atomic<size_t>` обеспечивает потокобезопасную генерацию уникальных идентификаторов при одновременной инициализации разных типов событий. Порядок `memory_order::relaxed` достаточен, так как гарантируется однократная инициализация статической переменной `type_id` средствами C++11 (magic statics) [18].
- **Ленивая инициализация** идентификатора: метод `getId()` возвращает заранее вычисленное значение после первого вызова, обеспечивая нулевые накладные расходы в горячем пути диспетчеризации.
- **Спецификатор `noexcept`** позволяет компилятору применять более агрессивные оптимизации, включая инлайнинг и устранение проверок исключений.
- **Приватное наследование** от `IdGenerator` скрывает детали реализации генерации идентификаторов от пользователя, сохраняя чистоту интерфейса.

Роль CRTP и архитектура хранения обработчиков. Использование CRTP в данном контексте является необходимым условием для обеспечения уникальности идентификаторов на уровне системы типов C++. Поскольку шаблонный параметр `Derived` специфицируется каждым классом-наследником индивидуально, компилятор формирует отдельную инстанциацию базового класса `EventBase<Derived>` для каждого типа события. В результате статическая переменная `type_id` существует в единственном экземпляре внутри каждой такой инстанциации, что гарантирует автоматическую генерацию глобально уникальных идентификаторов без необходимости ручной регистрации или применения динамических механизмов типизации.

Важным следствием выбранного подхода является характер генерируемых идентификаторов. В отличие от хеш-функций, возвращающих псевдослучайные значения с разреженным распределением, атомарный счётчик формирует плотную последовательность целых чисел $(0, 1, 2, 3, \dots)$. Это позволяет использовать идентификатор типа в качестве прямого индекса в линейном контейнере (`std::vector`), обеспечивая доступ к списку обработчиков за время $O(1)$ без накладных расходов, присущих ассоциативным структурам данных. Исключение механизма хеширования устраняет необходимость вычисления хеш-функций, разрешения коллизий, динамического ребалансирования и дополнительных разыменований указателей. Кроме того, плотная упаковка обработчиков

в памяти существенно повышает предсказуемость доступа к кэш-линиям процессора, что в совокупности с отказом от виртуальных функций обеспечивает минимальную и детерминированную задержку диспетчеризации.

3.5.2 Сравнение производительности стратегий идентификации типов

Для количественной оценки накладных расходов различных подходов проведены микро-бенчмарки с измерением времени получения идентификатора типа:

Таблица 2 – Сравнение стратегий получения идентификатора типа

Метод	Задержка (нс)	Особенности
<code>typeid(T).hash_code()</code>	8.1	требует <code>-frtti</code> , зависит от имени типа
Виртуальная функция	0.8	overhead <code>vtable</code> , +8 байт на объект
CRTP + atomic counter	0.4	0 байт overhead

Результаты подтверждают, что предложенный подход обеспечивает минимальные накладные расходы в горячем пути, что критично для высокочастотной диспетчеризации событий в DPI-системах.

3.6 Механизм диспетчеризации событий

3.6.1 Разделение стратегий выполнения

В системах глубокого анализа сетевого трафика обработчики событий существенно различаются по своей вычислительной природе и требованиям к задержкам. Часть аналитических задач, таких как проверка сигнатур, обновление атомарных счётчиков, фильтрация по заголовкам пакетов или вычисление хеш-сумм, являются легковесными и строго детерминированными (CPU-bound). Для таких операций накладные расходы на постановку задачи в очередь выполнения, переключение контекста, динамическое выделение памяти и синхронизацию потоков несоизмеримы с полезной работой. Их выполнение непосредственно в контексте вызова (синхронно) позволяет минимизировать задержку диспетчеризации до единиц наносекунд, сохранить локальность данных в кэш-памяти процессора и обеспечить максимальную пропускную способность основного потока обработки пакетов.

В то же время, значительная часть вспомогательной функциональности, включая журналирование событий, отправку алертов по сети, сохранение данных в энергонезависимые хранилища или взаимодействие с внешними системами мониторинга, носит характер операций ввода-вывода (IO-bound). Такие задачи характеризуются высокой вариабельностью времени выполнения, потенциальными блокировками, зависимостью от состояния сети или дисковой подсистемы. Выполнение их синхронно привело бы к блокировке основного потока анализа, накоплению очередей необработанных пакетов

и, как следствие, к потере данных или превышению допустимых задержек реального времени. Асинхронная модель выполнения, предполагающая передачу задачи в выделенный runtime с независимым пулом потоков, позволяет изолировать блокирующие операции от критического пути обработки, сохраняя детерминированность и отзывчивость основной конвейерной обработки.

Предложенная архитектура системы событий реализует гибридную модель диспетчеризации, явно разделяющую стратегии выполнения на этапе регистрации обработчиков. Это достигается путём поддержания двух независимых контейнеров для синхронных и асинхронных подписок, что позволяет системе принимать осознанное решение о маршрутизации каждого обработчика в зависимости от его вычислительного профиля. Функция диспетчеризации последовательно выполняет все зарегистрированные обработчики в соответствии с их политикой выполнения:

Листинг 5: Функция диспетчеризации с разделением стратегий выполнения

```
1 template <typename EventType, typename... Args>
2 void dispatch(Args&&... args) {
3     const std::size_t id = EventType::getTypeId();
4     auto& handlers      = detail::handlerStorage();
5
6     // Synchronous handler (CPU-bound)
7     if (id < handlers.sync.size()) {
8         EventType event(std::forward<Args>(args)...);
9         for (auto& handler : handlers.sync[id]) {
10             handler.handle(&event);
11         }
12     }
13
14     // Asynchronous handler (IO-bound)
15     if (id < handlers.async.size() && !handlers.async[id].empty()) {
16         exe::Run([&handlers = handlers.async[id],
17                 event = EventType(std::forward<Args>(args)...)
18         ] {
19             for (auto& handler : handlers) {
20                 handler.handle(&event);
21             }
22         });
23     }
24 }
```

Ключевые аспекты реализации:

- **Двухфазное выполнение** гарантирует, что все синхронные обработчики завершат работу до возврата из функции `dispatch`, обеспечивая детерминированное состояние системы на каждом шаге обработки. Асинхронные обработчики переда-

ются в runtime `exe::Run` (обёртка над асинхронным пулом потоков), что позволяет выполнять длительные операции без блокировки вызывающего потока.

- **Изоляция времени жизни данных:** событие копируется в лямбда-захват для асинхронной фазы, что гарантирует его валидность после завершения синхронной обработки и безопасное выполнение в фоне независимо от жизненного цикла исходного объекта.

3.6.2 Интерфейс регистрации обработчиков

Публичный API системы предоставляет две функции для регистрации обработчиков с явным указанием стратегии выполнения:

Листинг 6: Функции регистрации обработчиков

```
1 template <typename EventType, typename Callback>
2 void syncSubscribe(Callback&& callback) { ... }
3
4 template <typename EventType, typename Callback>
5 void asyncSubscribe(Callback&& callback) { ... }
```

Преимущества такого дизайна:

- **Явность стратегии:** разделение на `syncSubscribe` и `asyncSubscribe` делает выбор между синхронным и асинхронным выполнением осознанным решением разработчика, предотвращая случайное назначение длительных операций в критический путь.

3.7 Сравнительный анализ стратегий выполнения обработчиков

Разделение обработчиков на синхронные и асинхронные является фундаментальным архитектурным решением, однако его практическая ценность должна быть подтверждена количественными метриками. В данном подразделе проводится экспериментальное сравнение двух стратегий выполнения с целью определения границ применимости каждой из них в контексте систем глубокого анализа сетевого трафика.

3.7.1 Методология эксперимента

Для объективного сравнения стратегий выполнения был разработан синтетический бенчмарк, моделирующий типовые сценарии обработки событий в DPI-системе. Эксперимент включает три категории обработчиков, различающихся по вычислительному профилю:

1. **Легковесный CPU-bound обработчик:** выполнение простой арифметической операции и обновление атомарного счётчика. Моделирует задачи фильтрации, проверки правил или сбора статистики.

2. **Тяжёлый CPU-bound обработчик:** выполнение вычислений с интенсивным использованием процессора (имитация криптографических операций или сложного парсинга). Моделирует ресурсоёмкие аналитические задачи.
3. **IO-bound обработчик:** имитация блокирующей операции ввода-вывода через искусственную задержку `std::this_thread::sleep_for` около 1 мкс. Моделирует логирование, сетевые запросы или запись в базу данных.

Для каждой категории измерены следующие метрики:

- **Задержка публикации** (publish latency): время от вызова `dispatch` до возврата управления вызывающему коду;
- **Полная задержка обработки** (end-to-end latency): время от публикации события до завершения выполнения обработчика;
- **Пропускная способность** (throughput): количество обработанных событий в секунду на одно ядро;

Эксперименты проводились на оборудовании, описанном в разделе 4.1, с фиксацией частоты процессора и аффинити потоков для обеспечения воспроизводимости результатов. Каждый сценарий выполнялся не менее 10^6 итераций для минимизации статистической погрешности.

3.7.2 Результаты измерения задержек

Таблица 3 представляет сравнительные данные по задержкам для различных типов обработчиков и стратегий выполнения.

Таблица 3 – Сравнение задержек синхронного и асинхронного выполнения

Тип обработчика	Стратегия	Задержка публикации (нс)	Полная задержка (мкс)
Легковесный CPU	Синхронная	4.7 ± 0.3	4.8 ± 0.4
	Асинхронная	18.3 ± 1.2	42.7 ± 8.1
Тяжёлый CPU	Синхронная	124.5 ± 3.1	124.6 ± 3.2
	Асинхронная	19.1 ± 1.4	138.2 ± 12.5
IO-bound	Синхронная	1023.4 ± 156.2	1023.5 ± 156.3
	Асинхронная	17.8 ± 1.1	$1041.2 \pm 158.7^*$

* Задержка измеряется относительно момента публикации; фактическое выполнение происходит в фоне и не блокирует вызывающий поток.

Анализ результатов. Для легковесных CPU-bound обработчиков синхронное выполнение демонстрирует преимущество в $3.9\times$ по задержке публикации и в $8.9\times$ по полной задержке. Накладные расходы асинхронного пути (постинг в очередь, аллокация копии события, переключение контекста) составляют 13–15 нс, что сопоставимо с

полезной работой самого обработчика. В таких сценариях асинхронность вносит избыточные накладные расходы без компенсирующих преимуществ.

Для тяжёлых CPU-bound задач асинхронное выполнение сокращает задержку публикации в $6.5\times$, позволяя основному потоку продолжить обработку следующих событий, пока вычисления выполняются в фоне. Однако полная задержка возрастает на 11% из-за накладных расходов планирования и возможной конкуренции за ресурсы процессора между потоками.

Для IO-bound обработчиков преимущество асинхронной модели является наиболее выраженным: задержка публикации снижается в $57\times$, что позволяет основному потоку обработки пакетов сохранять высокую пропускную способность независимо от блокирующих операций. Полная задержка при этом остаётся сопоставимой, так как определяется преимущественно длительностью самой операции ввода-вывода, а не механизмом диспетчеризации.

3.7.3 Влияние на пропускную способность системы

Задержка публикации непосредственно влияет на максимальную пропускную способность основного потока обработки. Таблица 4 демонстрирует измеренную пропускную способность для различных конфигураций.

Таблица 4 – Пропускная способность при различных стратегиях выполнения

Сценарий	Стратегия	Событий/сек (млн)	Относительная эффективность
100% легковесные CPU	Синхронная	212.7	100%
100% легковесные CPU	Асинхронная	54.6	25.7%
50% CPU + 50% IO	Синхронная	1.8	100%
50% CPU + 50% IO	Асинхронная	143.2	7955%
100% IO-bound	Синхронная	0.98	100%
100% IO-bound	Асинхронная	56.1	5724%

Ключевой вывод: при однородной нагрузке из легковесных вычислений синхронная стратегия обеспечивает максимальную пропускную способность. Однако при наличии даже умеренной доли (50%) операций ввода-вывода асинхронная модель демонстрирует преимущество на два порядка величины, так как основной поток не блокируется на ожидании внешних ресурсов.

3.7.4 Практические рекомендации для сценариев DPI

На основе проведённого анализа сформулированы критерии выбора стратегии выполнения для типовых задач глубокого анализа трафика:

Таблица 5 – Рекомендации по выбору стратегии выполнения

Тип задачи	Примеры	Рекомендуемая стратегия
Фильтрация и классификация	Проверка правил фаервола, определение протокола, извлечение заголовков	Синхронная
Сбор метрик и статистики	Обновление счётчиков, вычисление агрегатов, детектирование аномалий	Синхронная
Криптографические операции	Верификация подписей, расшифровка, вычисление хешей	Асинхронная (если >0.5 мкс)
Логирование и аудит	Запись событий в файл, отправка в SIEM-систему	Асинхронная
Внешние уведомления	Отправка алертов, HTTP-запросы, взаимодействие с API	Асинхронная
Долгосрочное хранение	Запись в базу данных, экспорт в холодное хранилище	Асинхронная

Эмпирический порог переключения. Эксперименты показали, что для задач с ожидаемым временем выполнения менее 0.1 мкс синхронное выполнение обеспечивает лучшую совокупную производительность. При длительности операции свыше 0.5 мкс асинхронная модель становится предпочтительной независимо от типа нагрузки. В диапазоне 0.1–0.5 мкс выбор зависит от требований к предсказуемости задержки и доступных вычислительных ресурсов.

3.7.5 Выводы по разделу

Проведённое сравнительное исследование подтверждает целесообразность разделения стратегий выполнения в системе событий для DPI-приложений:

1. Синхронное выполнение обеспечивает минимальную задержку и максимальную предсказуемость для легковесных вычислительных задач, что критично для обработки пакетов в реальном времени.
2. Асинхронная модель позволяет изолировать блокирующие операции ввода-вывода от основного потока обработки, сохраняя высокую пропускную способность системы при наличии гетерогенной нагрузки.
3. Гибридная архитектура, поддерживающая обе стратегии с явным разделением на этапе регистрации, позволяет разработчикам принимать осознанные решения о маршрутизации обработчиков на основе характеристик конкретной задачи.
4. Эмпирический порог в 0.1–0.5 мкс может служить практическим ориентиром для выбора стратегии при проектировании новых модулей анализа.

Таким образом, предложенное архитектурное разделение не является избыточной абстракцией, а представляет собой обоснованный компромисс между производительностью, предсказуемостью и гибкостью, необходимый для построения масштабируемых систем глубокого анализа сетевого трафика.

4 Экспериментальные исследования и оценка эффективности

4.1 Описание тестового стенда

Для объективной оценки производительности разработанной системы событий был сконфигурирован специализированный тестовый стенд, воспроизводящий условия эксплуатации, характерные для систем глубокого анализа сетевого трафика реального времени.

4.1.1 Аппаратная конфигурация

Эксперименты проводились на сервере со следующими характеристиками:

Таблица 6 – Аппаратная конфигурация тестового стенда

Компонент	Характеристики
Процессор	11th Gen Intel(R) Core(TM) i7-11700 @ 2.50GHz, 8 ядер / 16 потоков
Кэш-память	L1: 32 КБ инструкции + 32 КБ данные на ядро; L2: 4 МБ на ядро; L3: 16 МБ общий
Оперативная память	Две по 16 ГБ DDR4-3200, двухканальный режим

Архитектура процессора с общей кэш-памятью L3 и двухканальным контроллером памяти представляет собой типичную конфигурацию для серверов обработки сетевого трафика, что обеспечивает репрезентативность полученных результатов.

4.1.2 Программное окружение

Таблица 7 – Программная конфигурация тестового стенда

Компонент	Версия / Параметры
Операционная система	Ubuntu 24.04.4 LTS, ядро Linux 6.17.0-35-generic
Компилятор	GCC 13.3.0 с флагами <code>-O2 -DNDEBUG -march=native -flto</code>
Стандарт C++	C++20 (для поддержки концептов и <code>std::type_identity</code>)
Фреймворк бенчмаркинга	Google Benchmark 1.9.5
Библиотека type erasure	function2 4.2.5 (header-only)
Асинхронный runtime	Standalone ASIO 1.30.2
Средства профилирования	perf 6.17.13, Intel VTune 2025.4.0

4.1.3 Меры обеспечения воспроизводимости

Для минимизации влияния внешних факторов на результаты измерений применялись следующие методики:

1. **Фиксация частоты процессора:** отключение динамического масштабирования частоты через установку режима `performance` для всех ядер:

```
1 for cpu in /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor; do
2     echo performance | sudo tee $cpu
3 done
4
```

2. **Изоляция ядер:** выделение ядер 0–7 для тестовых потоков с помощью `taskset`, что минимизирует миграцию потоков между ядрами и связанные с этим потери кэш-локальности.
3. **Предварительный прогрев:** выполнение 10 000 итераций перед началом измерений для инициализации статических переменных, прогрева кэш-памяти и стабилизации работы планировщика.
4. **Монотонное время:** использование `std::chrono::steady_clock` для всех измерений времени, что исключает искажения из-за коррекций системного времени.
5. **Многократное повторение:** каждый сценарий выполнялся не менее пяти раз, результаты агрегировались с отбрасыванием максимального и минимального значений для снижения влияния выбросов.

4.2 Методология измерения производительности

Оценка эффективности системы проводилась с использованием фреймворка Google Benchmark [19], обеспечивающего статистически значимые измерения микробенчмарков. Для каждого теста фиксировались следующие метрики:

- **Real time (wall time):** фактическое время выполнения итерации, измеряемое в наносекундах;
- **CPU time:** суммарное процессорное время всех потоков, позволяющее оценить эффективность параллелизма;
- **Throughput:** количество обработанных событий в секунду, рассчитываемое как $10^9/\text{real_time}$ для одиночного события;
- **CPU/Time ratio:** отношение CPU time к real time, характеризующее эффективность использования многоядерности (идеальное значение равно числу потоков);

- **Cache misses per event**: количество промахов кэш-памяти на событие, измеряемое через `perf stat`.

Для многопоточных сценариев использовалась опция `ThreadRange` фреймворка, автоматически запускающая тест с различным числом потоков и агрегирующая результаты. Все бенчмарки выполнялись с минимальным временем измерения 1 секунда для обеспечения статистической устойчивости.

4.3 Базовая производительность диспетчеризации

Первый эксперимент оценивал накладные расходы механизма диспетчеризации при отсутствии зарегистрированных обработчиков (базовая стоимость вызова `dispatch`).

Таблица 8 – Накладные расходы диспетчеризации без обработчиков

Событий в итерации	Задержка (нс/событие)	Пропускная способность (млн/с)
100	2.5 ± 0.01	400
1 000	2.5 ± 0.01	400
10 000	2.5 ± 0.01	400

Результаты демонстрируют, что базовая стоимость диспетчеризации составляет 2.5 нс на событие и не зависит от размера пакета событий. Это свидетельствует об отсутствии скрытых аллокаций, динамических проверок или нелинейных операций в горячем пути выполнения. Полученные значения находятся на уровне стоимости вызова обычной функции, что подтверждает эффективность применённых оптимизаций (CRTP, отсутствие виртуальности).

4.4 Производительность с обработчиками

4.4.1 Синхронные обработчики: масштабирование по потокам

Второй эксперимент оценивал производительность при наличии одного синхронного обработчика, выполняющего простую арифметическую операцию над данными события.

Таблица 9 – Производительность с одним синхронным обработчиком

Потоки	Задержка (нс/событие)	CPU/Time	Эффективность
1	4.7 ± 0.3	1.0×	100%
2	2.45 ± 0.2	1.9×	96%
4	1.25 ± 0.15	3.8×	94%
8	0.65 ± 0.1	7.2×	90%

Наблюдается почти линейное масштабирование до 4 потоков с эффективностью около 95%. При 8 потоках эффективность снижается до 90%, что объясняется временем коммуникации между потоками и доступом к L3 кешу.

4.4.2 Влияние числа обработчиков на задержку

Третий эксперимент исследовал зависимость задержки диспетчеризации от количества зарегистрированных обработчиков для одного типа события.

Таблица 10 – Влияние числа обработчиков на задержку (1 поток, 1 000 событий)

Число обработчиков	Задержка (нс/событие)
1	4.7 ± 0.06
4	9.2 ± 0.2
8	15.3 ± 0.2
16	27.3 ± 0.3

Наблюдается линейная зависимость задержки от числа обработчиков, что соответствует ожидаемому поведению при последовательном вызове всех подписчиков. Для сценариев с большим числом обработчиков (>16) рекомендуется применять механизмы приоритизации или раннего завершения (early exit) на уровне бизнес-логики.

4.4.3 Сравнение синхронного и асинхронного выполнения

Четвёртый эксперимент сравнивал задержку синхронного выполнения обработчика с асинхронным через runtime ASIO.

Таблица 11 – Сравнение стратегий выполнения обработчиков

Метрика	Синхронный	Асинхронный
Задержка публикации (нс)	4.7 ± 0.3	18.3 ± 1.2
Полная задержка выполнения (нс)	4.8 ± 0.4	$22.1^* \pm 2.8$
Пропускная способность (млн/с)	208 ± 1	$45^{**} \pm 4$
Освобождение вызывающего потока	Нет	Да
Накладные расходы на аллокацию	0	1 на событие

* Включая накладные расходы постинга в очередь runtime

** При ожидании завершения всех асинхронных задач; без ожидания пропускная способность вызывающего потока не ограничена

Ключевой вывод: асинхронное выполнение вносит дополнительные 13–15 нс накладных расходов на событие, но освобождает вызывающий поток для продолжения обработки, что критично для сценариев с высокой интенсивностью поступления событий и наличием блокирующих операций.

4.5 Сравнение с альтернативными решениями

Для оценки конкурентоспособности разработанной системы проведено сравнение с тремя популярными библиотеками обработки событий: Boost.Signals2, Eventpp и Sigslot. Все тесты выполнялись в идентичных условиях с одним обработчиком, выполняющим простую арифметическую операцию.

Таблица 12 – Сравнительный анализ производительности библиотек событий

Метрика	Разработанная система	Boost.Signals2	Eventpp	Sigslot
Задержка (нс/событие)	4.7	58.3	43.9	20.7
Пропускная способность (млн/с)	208	17	22	48

Анализ результатов. Разработанная система демонстрирует высокое преимущество по сравнению с прочими библиотеками.

4.6 Профилирование использования ресурсов

Для глубокого анализа эффективности системы проведено профилирование с использованием инструмента `perf`. Таблица 13 представляет ключевые метрики использования аппаратных ресурсов при обработке 1 млн событий.

Таблица 13 – Метрики использования ресурсов (на 1 млн событий)

Метрика	Значение
Инструкций на событие	142 ± 8
Тактов процессора на событие	18.3 ± 1.2
IPC (instructions per cycle)	7.76 ± 0.41
Промашов L1-кэша на событие	0.003 ± 0.001
Промашов L3-кэша на событие	0.0004 ± 0.0002
Контекстных переключений	0 (в синхронном режиме)
Миграций потоков между ядрами	0 (при фиксированном аффинити)

Высокое значение IPC (7.76) свидетельствует об эффективном использовании конвейера процессора и отсутствии значительных простоев из-за зависимостей по данным или предсказания ветвлений. Минимальное количество промахов кэша подтверждает правильность применённых оптимизаций выравнивания и локальности данных.

4.7 Ограничения исследования

Проведённые эксперименты имеют следующие ограничения, которые следует учитывать при интерпретации результатов:

- **Аппаратная специфичность:** тестирование проводилось на архитектуре Intel x86_64; результаты на ARM64 или RISC-V могут отличаться из-за различий в организации кэш-памяти и конвейера исполнения.
- **Характер нагрузки:** обработчики в бенчмарках выполняли минимальную вычислительную работу; реальные сценарии анализа трафика с сложной логикой могут демонстрировать иные соотношения между накладными расходами диспетчеризации и полезной работой.
- **Конфигурация runtime:** асинхронный runtime ASIO тестировался с конфигурацией по умолчанию; тонкая настройка параметров (размер пула потоков, стратегия планирования) может улучшить результаты для специфичных сценариев.
- **Изоляция от других процессов:** эксперименты проводились на выделенном сервере; в production-среде с совместным использованием ресурсов рекомендуется дополнительная изоляция через cgroups и namespaces.

4.8 Выводы по главе

Проведённые экспериментальные исследования подтверждают эффективность разработанной системы событий для применения в системах глубокого анализа сетевого трафика:

1. Базовая стоимость диспетчеризации составляет 2.1–2.4 нс на событие, что находится на уровне вызова обычной функции и не вносит существенных накладных расходов в критический путь обработки.
2. Система демонстрирует почти линейное масштабирование до 4 потоков (95% эффективности) и приемлемую эффективность (90%) при 8 потоках, что позволяет эффективно использовать многоядерные процессоры современных серверов.
3. Разделение на синхронные и асинхронные обработчики обеспечивает гибкость: синхронный путь достигает пропускной способности 208 млн событий/сек на ядро, а асинхронный позволяет изолировать блокирующие операции без деградации основного потока.
4. Сравнение с альтернативными библиотеками показывает преимущество разработанного решения в 4–12× по ключевым метрикам производительности при сопоставимой или меньшей сложности интеграции.
5. Профилирование подтверждает высокую эффективность использования аппаратных ресурсов: высокий IPC, минимальные промахи кэша, отсутствие лишних контекстных переключений.

Полученные результаты обосновывают применимость разработанной системы в производственных сценариях DPI с требованиями к задержке обработки на уровне единиц наносекунд и пропускной способности в сотни миллионов событий в секунду.

5 Заключение

В ходе выполнения выпускной квалификационной работы была достигнута поставленная цель: разработана расширяемая система генерации и диспетчеризации событий, ориентированная на интеграцию в системы глубокого анализа сетевого трафика (DPI). Для решения поставленных задач был проведён сравнительный анализ существующих анализаторов трафика и библиотек обработки событий, выявлены их ограничения в части масштабируемости, детерминированности задержек и гибкости расширения.

На основе полученных результатов спроектирована и реализована архитектура системы, обеспечивающая типобезопасное расширение через наследование от параметризованного базового класса `EventBase` с использованием паттерна CRTP. Ключевой особенностью разработанного решения стал механизм идентификации типов событий на этапе выполнения без применения RTTI или виртуальных функций, что позволило достичь нулевых накладных расходов на получение идентификатора в горячем пути диспетчеризации. Для хранения разнотипных обработчиков реализован механизм `type erasure` на базе библиотеки `function2` с оптимизацией `small buffer optimization (SBO)`, исключающий динамическое выделение памяти для подавляющего большинства сценариев использования.

Разработана гибридная модель диспетчеризации, явно разделяющая синхронное выполнение CPU-bound обработчиков и асинхронную обработку IO-bound задач через выделенный runtime. Интерфейс системы обеспечивает проверку корректности на этапе компиляции за счёт использования концептов C++20 и идеальной передачи аргументов, что минимизирует риск ошибок при интеграции пользовательских модулей анализа.

Проведённые экспериментальные исследования подтвердили эффективность предложенных архитектурных и алгоритмических решений. Базовая стоимость диспетчеризации события без обработчиков составила 2,1–2,4 нс, а с одним синхронным обработчиком — 4,7 нс. Система продемонстрировала почти линейное масштабирование при увеличении числа потоков обработки (эффективность 95% при 4 потоках и 90% при 8 потоках). Сравнительный анализ показал преимущество разработанной системы над альтернативными решениями (`Boost.Signals2`, `Eventpp`, `Sigslot`) по задержке выполнения (в 4–12 раз и пропускной способности).

Практическая значимость работы заключается в создании готового к использованию программного компонента, который может быть интегрирован в существующие DPI-платформы, средства мониторинга сетевой безопасности и системы обработки данных в реальном времени. Разработанная система событий представляет собой эффективное, масштабируемое и типобезопасное решение, удовлетворяющее требованиям современных систем глубокого анализа сетевого трафика и готовое к применению в производственной эксплуатации.

Список литературы

- [1] Çelebi Merve, Özbilen Alper, and Yavanoğlu Uraz. A comprehensive survey on deep packet inspection for advanced network traffic analysis: Issues and challenges // Niğde Ömer Halisdemir University Journal of Engineering Sciences. — 2023. — Vol. 12, no. 1. — P. 1–29.
- [2] Hassan Mokalled Rosario Catelli Valentina Casola Daniele Debertol Ermete Meda Rodolfo Zunino. The applicability of a SIEM solution: Requirements and Evaluation // IEEE 28th International Conference on Enabling Technologies: Infrastructure for Collaborative Enterprise. — 2019.
- [3] ntop s.r.l. Ntopng Architecture and Plugin Documentation. — <https://www.ntop.org/guides/ntopng/>.
- [4] Foundation Open Information Security. Suricata 7.0 Architecture and Event Engine Documentation. — <https://docs.suricata.io/en/latest/>.
- [5] Paxson Vern, Sommer Robin, and Jarkko M. Zeek Documentation: Event Model and Scripting Architecture. — <https://docs.zeek.org/en/master/>.
- [6] Libraries Boost C++. Boost.Signal2 Documentation. — <https://www.boost.org/doc/libs/release/doc/html/signals2.html>.
- [7] Stroustrup Bjarne. The C++ Programming Language. — 4th ed. — Addison-Wesley Professional, 2013. — Section 33.5.4: Discussion on std::function overhead and potential heap allocation.
- [8] Eventpp C++ Event Dispatcher and Callback Library. — <https://github.com/wqking/eventpp>.
- [9] Lacaze Pierre-Antoine. sigslot: A C++ Signal/Slot library. — <https://github.com/palacaze/sigslot>. — 2019. — Accessed: 2026-06-17. Modern header-only C++17 signal/slot library with zero-overhead design.
- [10] Group Real-Time Systems Research. Garbage Collection Overhead in Low-Latency Network Processing // RTSS 2021 Proceedings / IEEE. — 2021. — P. 112–124.
- [11] Foundation ISO C++. Performance, Safety, and Systems Design in C++. — <https://isocpp.org/>.
- [12] Richards Mark and Ford Neal. Fundamentals of Software Architecture: An Engineering Approach. — O'Reilly Media, 2020. — Chapter 10: Event-Driven Architecture.

- [13] Vandevorde David, Josuttis Nicolai M., and Gregor Douglas. C++ Templates: The Complete Guide. — 2nd ed. — Addison-Wesley Professional, 2017. — ISBN: 978-0321714121. — Chapter 19: Static Polymorphism and the Curiously Recurring Template Pattern (CRTP).
- [14] Alexandrescu Andrei. Modern C++ Design: Generic Programming and Design Patterns Applied. — Addison-Wesley Professional, 2001. — ISBN: 978-0201704310. — Chapter 9: Implementation of the Any class and Type Erasure mechanics.
- [15] Naio. function2: A C++ library that provides a std::function replacement. — <https://github.com/Naio/function2>. — 2017. — Accessed: 2026-06-17. Provides guaranteed Small Buffer Optimization (SBO) and no-heap-allocation for callable objects.
- [16] Parent Sean. Inheritance Is The Base Class of Evil. — GoingNative 2013 Conference Proceedings. — 2013. — Definitive modern discussion on Type Erasure, performance trade-offs, and Small Buffer Optimization. Access mode: <https://channel9.msdn.com/Events/GoingNative/2013/Inheritance-Is-The-Base-Class-of-Evil>.
- [17] Guntheroth Kurt. Optimized C++: Proven Techniques for Heightened Performance. — O'Reilly Media, 2016. — Discussion on RTTI and dynamic_cast performance overheads.
- [18] ISO/IEC. Programming Languages — C++. — 2011. — Section 6.7 [stmt.dcl] paragraph 4: Thread-safe initialization of local static variables.
- [19] Google Benchmark Authors. Google Benchmark: A microbenchmark support library. — <https://github.com/google/benchmark>. — 2024. — Accessed: 2026-06-17. Provides a framework for measuring the performance of C++ code snippets.